

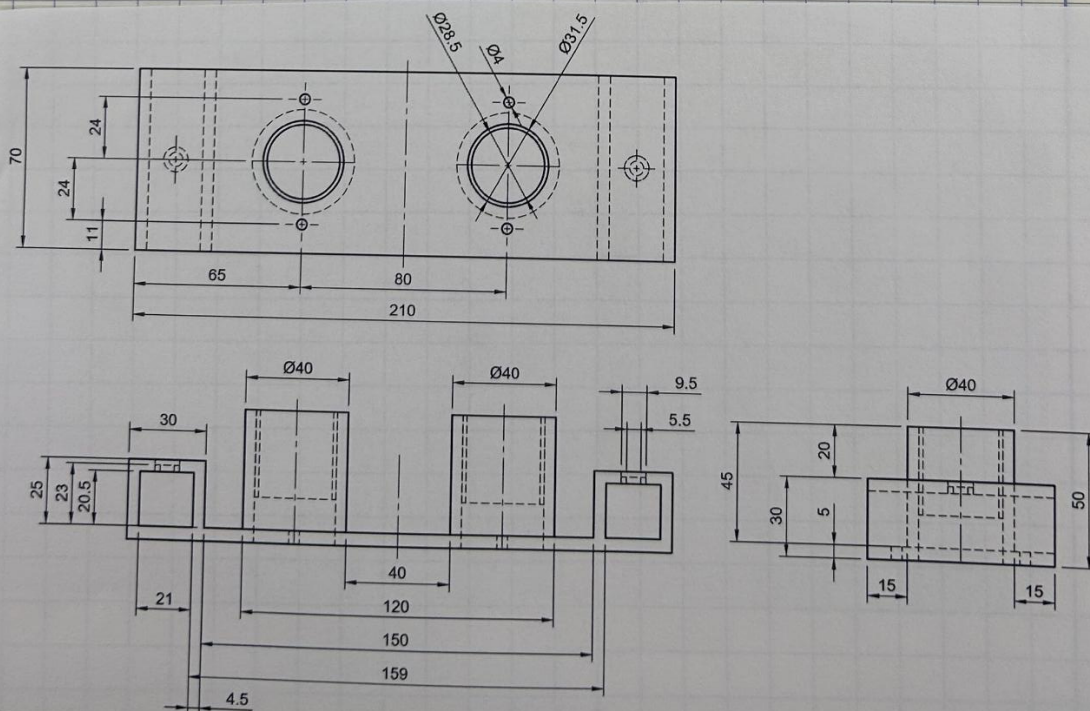
EZO-PMP

5/8/25 3

For this project we will dynamically calibrate a rain tipper. Ok now ~~lets~~ go over the two technical terms I just used. First, what is a rain tipper? It is a device used to measure rain with a tipping spoon that fills after a certain amount of rain falls in it. Once it fills the spoon dumps out and since there is an electronic circuit inside it outputs a simple digital signal to read. Our Next definition is dynamic calibration, this is the process of calibrating during operation. To do this we will measure the tip rate of the spoon by reading the digital signal as we change the water's intensity from our EZO-PMP's.

Ok what are these EZO-PMP's. They are Atlas Scientific's Peristaltic Pump that has UART & I2C capabilities.

Before I started programming the pumps I made a 3D print to hold them. The print was used to connect a simple open-builds (sorry but this company ~~reading~~ ~~down~~ business) we used their V-slot 20x20 linear rail to make a simple frame for our 3D print to be ~~be~~ held 1-2 feet off the table. Below is a drawing of our 3D print



5/28/25 5

Using UART with the EZO-PMP

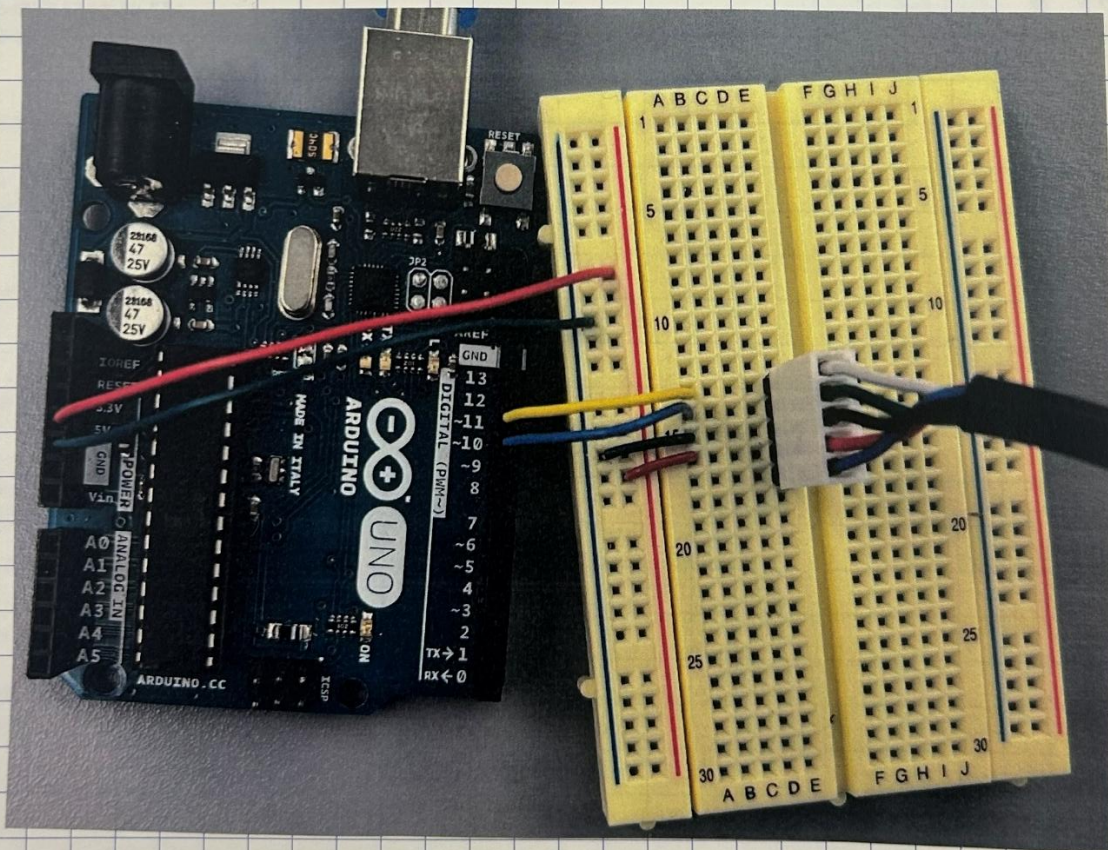
Let's setup the circuit we need. Here's the diagram below



This is the ~~chord~~ chord output for the pump

Now you might be wondering what pin 10 & 11 are they are the pins we will encode on our arduino uno to be RX & TX (remember for the arduino RX will be pin 10 & TX is pin 11) (for uart they are flip-flopped from the ~~device~~ device and micro controller)

Below is an image of the circuit I made and used



5/28/25

7

Now lets write some code for our pump for UART

```
#include <SoftwareSerial.h>
#include <Wire.h>
const byte rxPin = 10;
const byte txPin = 11;

SoftwareSerial mySerial(rxPin, txPin);

String inputString = "";
String sensorString = "";
bool inputStringComplete = false;
bool sensorStringComplete = false;

void setup()
{
  Serial.begin(9600);
  mySerial.begin(9600);
  inputString.reserve(10);
  sensorString.reserve(30);
  Serial.println("Starting loop...");
}

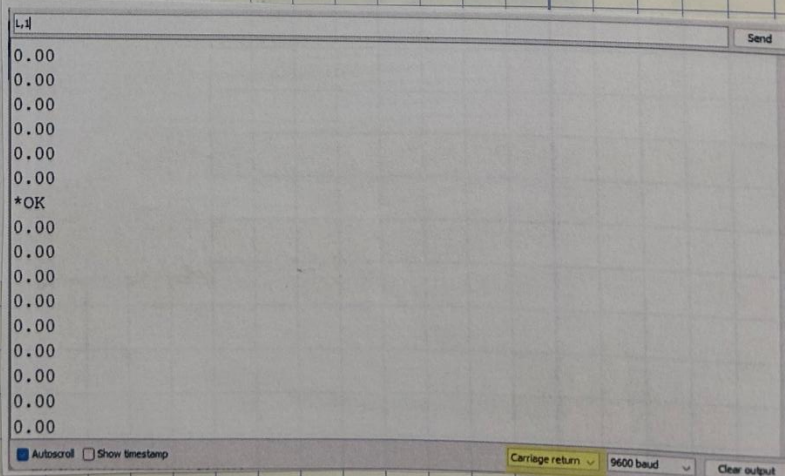
void loop()
{
  if (inputStringComplete == true) //Send commands to sensor board
  {
    mySerial.print(inputString);
    mySerial.print('\r');
    inputString = "";
    inputStringComplete = false;
  }

  if (mySerial.available() > 0) //Receive data from sensor
  {
    char inChar = (char)mySerial.read();
    sensorString += inChar;
    if (inChar == '\r')
    {
      sensorStringComplete = true;
    }
  }

  if (sensorStringComplete == true)
  {
    Serial.println(sensorString);
    sensorString = "";
    sensorStringComplete = false;
  }
}

void serialEvent()
{
  inputString = Serial.readStringUntil(13);
  inputStringComplete = true;
}
```

This code will allow us to use UART to communicate to the EZO-PMP. Page 9 will have a list of functions to use. To push functions you will have to go to the serial monitor. !!!In the serial monitor be sure to carriage return it is highlighted below!!! Also if you look at the image of the serial monitor you'll see the ok thats when I told the light to shut off for more info see page 11. Also if you see the top where the input bar is you can see that I am about to turn the light on for the function see the next page. for what happened see page 11.



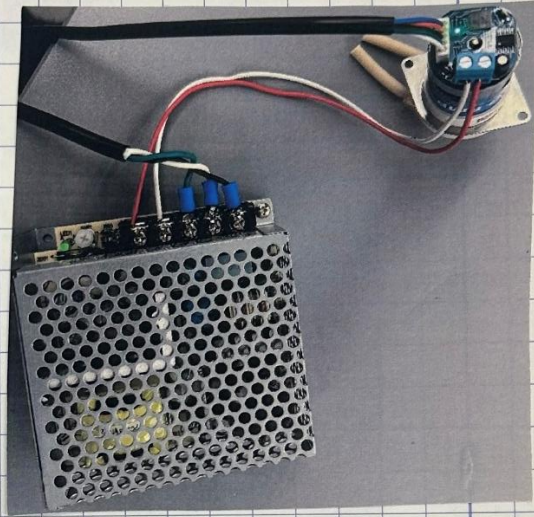
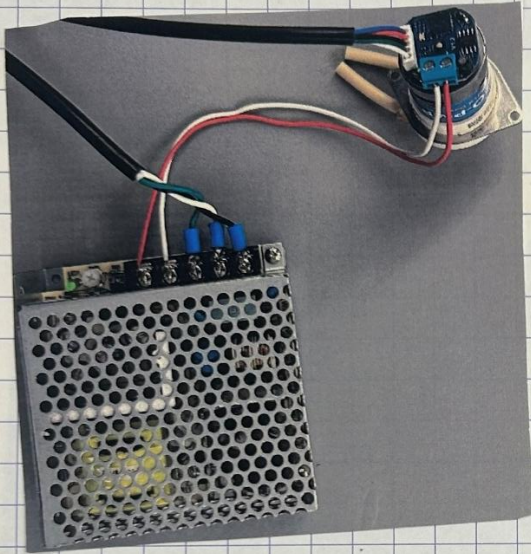
5/28/25 9

Ok since we're in UART mode here's all the functions we can use with it. I2C will be the most important one but we'll work with the light next page to be sure we know what we're doing.

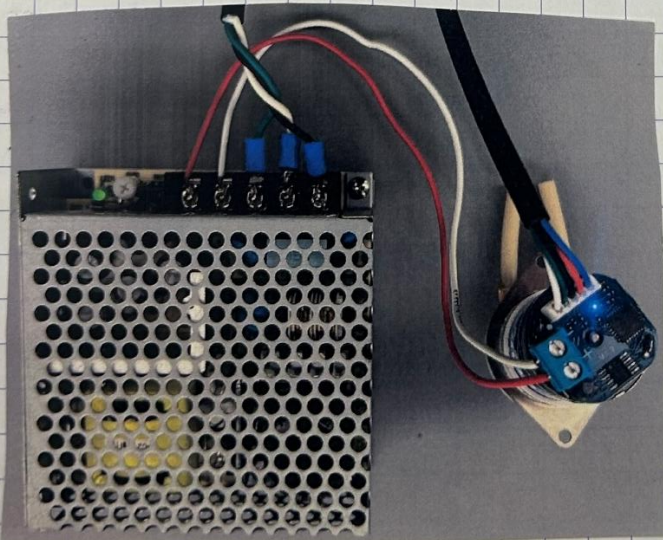
All commands are ASCII strings or single ASCII characters.		
Command	Function	
Baud	change baud rate	pg. 39
C	enable/disable continuous mode	pg. 18
Cal	performs calibration	pg. 31
D	dispense modes	pg. 20 - 26
Factory	enable factory reset	pg. 41
Find	finds device with blinking white LED	pg. 17
i	device information	pg. 35
Invert	invert dispensing direction	pg. 29
I2C	change to I ² C mode	pg. 42
L	enable/disable LED	pg. 16
Name	set/show name of device	pg. 34
O	enable/disable parameters	pg. 32
P	pause dispensing	pg. 27
Plock	enable/disable protocol lock	pg. 40
Pv	check pump voltage	pg. 33
R	returns a single reading	pg. 19
Sleep	enter sleep mode/low power	pg. 38
Status	retrieve status information	pg. 37
Tv	total volume dispensed	pg. 30
X	stop dispensing	pg. 28
*OK	enable/disable response codes	pg. 36

5/29/25 11

Alright lets use some functions. First off lets go simple with the On-pump light. Ok at first we have to be sure our serial monitor looks like the image on page 7 then type `L,0` to shut the light off then hit enter. next we'll type `L,1` to turn the light back on. Note the light will ~~be~~ flicker as we run our code. And the light is green for UART mode.



Ok now for the most important part of using UART we will switch the pump to I2C mode to do this we will type `I2C,104` to make our address `0x68` For our other pump we want its address to be `0x67` so we will type `I2C,103`.



As seen here I2C mode has a blue light instead of green.

The steel box is a $\pm 12V$ DC power supply (RS-50-12) meanwell 4.2A

5/29/25

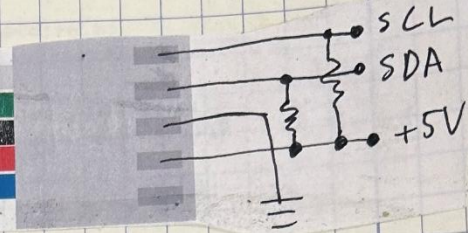
13

Now we'll ~~start~~ start using the pumps in I2C mode.
we'll keep them like this for the rest of the project.

Here's our new circuit:

White - RX/SCL
Green - TX/SDA
Black - GND
Red - VCC
Blue - INT

Data and power cable pinout



Note the resistors are 4.7K's

Now here's the commands in I2C mode

All commands are ASCII strings or single ASCII characters.

Command	Function	
Baud	switch back to UART mode	pg. 75
Cal	performs calibration	pg. 65
D	dispense modes	pg. 54 - 60
Factory	enable factory reset	pg. 74
Find	finds device with blinking white LED	pg. 52
i	device information	pg. 69
Invert	invert dispensing direction	pg. 63
I2C	change I ² C address	pg. 73
L	enable/disable LED	pg. 51
Name	set/show name of device	pg. 68
O	enable/disable parameters	pg. 66
P	pauses the pump during dispensing	pg. 61
Plock	enable/disable protocol lock	pg. 72
Pv	check pump voltage	pg. 67
R	returns a single reading	pg. 53
Sleep	enter sleep mode/low power	pg. 71
Status	retrieve status information	pg. 70
Tv	total volume dispensed	pg. 64
X	stop dispensing	pg. 62

5/29/15

15

Now that we're using I²C we want to change the state of our light as you can see on the last page the function to do this is L to turn it ~~off~~ on is a 1 while off is 0

Here's some code that will turn the light off & on repeating for sections of 5 seconds

```
#include <Wire.h>

int add = 0x67;

void setup()
{
    Wire.begin();
}

void loop()
{
    Wire.beginTransaction(add);    //light off
    Wire.write(0x4C)://L
    Wire.write(0x2C)://S
    Wire.write(0x30)://0
    Wire.endTransmission();
    delay(5000);

    Wire.beginTransaction(add);    //light on
    Wire.write(0x4C)://L
    Wire.write(0x2C)://S
    Wire.write(0x31)://1
    Wire.endTransmission();

    delay(5000);
}
```

```
#include <Wire.h>

int add = 0x67;

void setup()
{
    Wire.begin();
    Wire.beginTransaction(add);
    Wire.write(0x58)://X
    Wire.endTransmission();
    Wire.beginTransaction(add);
    Wire.write(0x44)://0
    Wire.write(0x43)://C
    Wire.write(0x2C)://S
    Wire.write(0x35)://5
    Wire.write(0x30)://0
    Wire.write(0x2C)://S
    Wire.write(0x31)://1
    Wire.endTransmission();
}
```

At this point we'll do some basic pumping which will use the 3D cadded piece and the openbuilds frame to pump • 50 mL in one minute

The X is to tell the pump to stop any process under way

!!! Remember you need void loop(){} at the end or you'll get an error!!!